

OFFENSIVE SECURITY TOOLS

VERA DEVELOPER

Contenido

OFFENSIVE SECURITY TOOLS	1
 Glosario	4
 Introducción	5
 Características principales	5
 Estructura del proyecto	6
 Objetivos del Proyecto	6
Objetivo General	6
Objetivos Específicos	6
 Estructura del Proyecto	7
Enfoque Arquitectónico: POO + MVC	7
 Paradigma: Programación Orientada a Objetos (POO)	7
 Patrón de Diseño: Modelo - Vista - Controlador (MVC)	8
 Relación con Django	8
Levantamiento de Requisitos Técnicos	¡Error! Marcador no definido.
1. Requisitos Funcionales (RF)	¡Error! Marcador no definido.
RF-01: Despliegue e Instalación	¡Error! Marcador no definido.
RF-02: Escaneo de Red y Seguridad del Servidor	¡Error! Marcador no definido.
RF-03: Auditoría de Dependencias y Vulnerabilidades	¡Error! Marcador no definido.
RF-04: Evaluación de Configuraciones y Riesgos del Sistema	¡Error! Marcador no definido.
RF-05: Reportes y Comunicación	¡Error! Marcador no definido.
RF-06: Arquitectura y Desarrollo	¡Error! Marcador no definido.
RF-07: Mantenimiento y Extensibilidad	¡Error! Marcador no definido.
2. Requisitos No Funcionales (RNF)	¡Error! Marcador no definido.
RNF-01: Seguridad	¡Error! Marcador no definido.
RNF-02: Rendimiento y Escalabilidad	¡Error! Marcador no definido.
RNF-03: Usabilidad y Operabilidad	¡Error! Marcador no definido.

RNF-04: Portabilidad y Compatibilidad	¡Error! Marcador no definido.
RNF-05: Mantenibilidad.....	¡Error! Marcador no definido.
3. Requisitos Técnicos Detallados	¡Error! Marcador no definido.
4. Restricciones y Exclusiones.....	¡Error! Marcador no definido.
 Casos de Uso - Offensive Security Tools	¡Error! Marcador no definido.
Caso de Uso 1: Instalación y Configuración Inicial	¡Error! Marcador no definido.
Caso de Uso 2: Ejecución Automática del Escaneo y Generación de Reportes	¡Error! Marcador no definido.
Caso de Uso 3: Escaneo Manual y Análisis Puntual.....	¡Error! Marcador no definido.
Caso de Uso 4: Auditoría de Dependencias	¡Error! Marcador no definido.
Caso de Uso 5: Escaneo y Detección de Servicios y Puertos	¡Error! Marcador no definido.
Caso de Uso 6: Evaluación de Configuraciones Inseguras	¡Error! Marcador no definido.
Caso de Uso 7: Generación y Formateo de Reportes PDF.....	¡Error! Marcador no definido.
Caso de Uso 8: Envío Automático de Reportes por Correo Electrónico	¡Error! Marcador no definido.
Caso de Uso 9: Actualización Manual de la Herramienta	¡Error! Marcador no definido.
Caso de Uso 10: Configuración Dinámica vía Variables de Entorno	¡Error! Marcador no definido.
 Casos De Uso UML	¡Error! Marcador no definido.
Actores:.....	¡Error! Marcador no definido.
Casos de Uso (Con descripciones breves):	¡Error! Marcador no definido.
Relaciones entre casos de uso:.....	¡Error! Marcador no definido.
Diagrama.....	¡Error! Marcador no definido.

Glosario

Término	Definición
Auditoría de dependencias	Proceso de análisis de los paquetes o librerías utilizadas por un proyecto (ej. pip, npm) para detectar vulnerabilidades conocidas asociadas a sus versiones.
CVE (Common Vulnerabilities and Exposures)	Sistema estandarizado para identificar y describir públicamente vulnerabilidades de software. Cada CVE tiene un identificador único (ej. CVE-2024-12345).
Django	Framework de desarrollo web en Python, usado aquí como base estructural sin activar su servidor web en la primera fase.
Escaneo de red / puertos	Técnica utilizada para identificar puertos abiertos y servicios disponibles en un sistema mediante herramientas como nmap.
python-nmap	Librería de Python que actúa como envoltorio (wrapper) del comando nmap, permitiendo su uso programático dentro del proyecto.
PDF (Portable Document Format)	Formato de archivo utilizado para generar reportes de resultados en forma legible y portable.
Pipenv	Herramienta de Python para gestionar entornos virtuales y dependencias, permitiendo reproducibilidad y aislamiento del entorno.
Pre-commit	Framework para ejecutar scripts antes de confirmar cambios en Git (commits), asegurando estándares de calidad, formato y validación del código.
POO (Programación Orientada a Objetos)	Paradigma de programación que organiza el código en clases y objetos, promoviendo modularidad, reutilización y escalabilidad.

Término	Definición
MVC (Modelo-Vista-Controlador)	Patrón de arquitectura que separa la lógica del negocio (controlador), los datos (modelo) y la presentación (vista), facilitando mantenimiento y extensión.
SMTP (Simple Mail Transfer Protocol)	Protocolo estándar para el envío de correos electrónicos. En este proyecto, se utiliza para distribuir los reportes generados.
Variable de entorno	Parámetro configurable del sistema operativo que permite modificar el comportamiento del programa sin editar el código fuente (ej. EMAIL, TIME).
Superficie de ataque	Conjunto de puntos en un sistema susceptibles de ser atacados o explotados por amenazas externas.
TLS/SSL	Protocolos criptográficos utilizados para proteger las comunicaciones (como el envío de correos) mediante cifrado seguro.



Introducción

Offensive Security Tool es una herramienta automatizada de análisis de seguridad diseñada para ser instalada directamente en los servidores de los clientes. Su objetivo principal es identificar riesgos, vulnerabilidades y configuraciones inseguras presentes tanto en los proyectos alojados como en el propio servidor.

La herramienta ejecuta escaneos de red, análisis de dependencias y evaluación de amenazas, generando un reporte en formato PDF con los hallazgos más relevantes. Este reporte es enviado automáticamente por correo electrónico con una frecuencia configurable por el cliente (diaria o semanal), permitiendo mantener una vigilancia constante y proactiva del entorno.



Características principales

- Instalación local en servidor del cliente: sin conexión a servidores externos.
- Configuración mediante variables de entorno (ej. TIME, EMAIL, PROJECT_PATHS).
- Escaneo de red y puertos usando python-nmap.

- Auditoría de dependencias de proyectos (pip, npm, etc.), con clasificación de vulnerabilidades (Crítica, Alta, Media).
- Detección de riesgos comunes del sistema (servicios expuestos, configuraciones inseguras).
- Generación automática de reportes en PDF.
- Envío automatizado de reportes por correo electrónico.
- Arquitectura limpia y modular, diseñada con Python y Django.
- Gestión de dependencias con Pipenv y control de calidad con pre-commit.

Estructura del proyecto

El código fuente está organizado para facilitar la escalabilidad, mantenimiento y separación de responsabilidades, siguiendo principios sólidos de arquitectura de software. Destacan carpetas como:

- `controllers/`: controladores para manejar la lógica principal (escaneos, reportes, autenticación, etc.)
- `services/`: escaneo de puertos, vulnerabilidades y exploits.
- `views/`: presentación de resultados en CLI o PDF.
- `models/`: representación estructurada de objetivos y resultados.
- `utils/`: herramientas auxiliares de red.

 **Nota:** En esta primera fase, las actualizaciones del sistema deben ser aplicadas manualmente por el cliente. Futuras versiones podrán incluir actualización remota, panel web y sincronización con plataformas externas.

Objetivos del Proyecto

Objetivo General

Desarrollar una herramienta automatizada, modular y extensible que permita analizar el estado de seguridad de servidores y proyectos alojados, generando reportes periódicos que ayuden a identificar y mitigar vulnerabilidades de forma proactiva.

Objetivos Específicos

1. Automatizar el análisis de seguridad en entornos productivos locales, sin necesidad de conexión a servicios externos.
2. Detectar servicios activos y puertos abiertos en el servidor mediante escaneo local con `python-nmap`.

3. Auditar las dependencias de los proyectos alojados, clasificando las vulnerabilidades según su nivel de criticidad (Crítica, Alta, Media).
4. Identificar configuraciones inseguras y servicios innecesarios que representen un riesgo potencial para la seguridad del servidor.
5. Generar reportes claros y estructurados en formato PDF, con un resumen detallado de los hallazgos de seguridad.
6. Permitir el envío automatizado de reportes vía correo electrónico con una frecuencia definida por el cliente (diaria o semanal), utilizando variables de entorno.
7. Diseñar una arquitectura limpia, modular y escalable, basada en Python y Django, que facilite futuras extensiones como paneles web, APIs o integración con otras herramientas de seguridad.
8. Garantizar buenas prácticas de desarrollo, mediante el uso de Pipenv para la gestión de entornos y pre-commit para asegurar la calidad del código.

Estructura del Proyecto

```

offensive_security_tool/
├── .github/           # Workflows para CI/CD
│   └── workflows/
├── .vscode/           # Configuración de entorno (opcional)
├── src/
│   ├── __init__.py
│   ├── controllers/
│   │   ├── __init__.py
│   │   ├── models/
│   │   │   ├── __init__.py
│   │   │   ├── views/
│   │   │   │   ├── __init__.py
│   │   │   ├── services/
│   │   │   │   ├── __init__.py
│   │   │   ├── utils/
│   │   │   │   ├── __init__.py
│   │   └── main.py
├── Pipfile             # Dependencias
├── Pipfile.lock         # Versiones bloqueadas
├── .pre-commit-config.yaml # Hooks de validación
├── setup.py             # Configuración de empaquetado
├── .gitignore           # Archivos ignorados por Git
├── README.md            # Este archivo
├── LICENSE.md           # Licencia MIT
├── SECURITY.md           # Política de seguridad
└── .python-version      # Versión requerida de Python
  
```

Enfoque Arquitectónico: POO + MVC

Paradigma: Programación Orientada a Objetos (POO)

El desarrollo de Offensive Security Tool se basa en principios de POO para mejorar la organización del código, facilitar la reutilización y garantizar la extensibilidad del sistema. Cada componente principal (escáner de puertos, generador de reportes, analizador de vulnerabilidades, etc.) se implementa como una clase con responsabilidades bien definidas.

Beneficios de este enfoque:

- Encapsulamiento de lógica por dominio (por ejemplo, PortScanner, DependencyAuditor, ReportGenerator).
- Facilidad para pruebas unitarias y mockeo de componentes.
- Mejor mantenimiento y comprensión del código.
- Base sólida para escalabilidad futura.

Patrón de Diseño: Modelo - Vista - Controlador (MVC)

El proyecto sigue una adaptación del patrón MVC, donde:

- Modelos (models/): Representan las entidades del sistema (objetivos, resultados de escaneo, vulnerabilidades). En versiones futuras podrían integrarse con Django ORM para persistencia.
- Controladores (controllers/): Orquestan la lógica de negocio y el flujo de datos entre servicios y vistas. Ejemplo: `scan_result_controller.py` coordina un escaneo y genera un objeto de resultado.
- Vistas (views/): Se encargan de presentar los resultados. En esta fase, pueden ser salidas por consola (CLI) o archivos PDF. Posteriormente, se puede agregar una vista web.

Este enfoque permite mantener una separación clara de responsabilidades y facilita:

- Aislar la lógica de seguridad y escaneo de la presentación.
- Escalar hacia otras interfaces (API REST, dashboard web) sin reescribir la lógica principal.
- Desarrollar e integrar nuevas funciones (como nuevos tipos de reportes o análisis) de forma modular.

Relación con Django

Aunque Django ya implementa una versión propia de MVC (llamada MTV: Model-Template-View), en este proyecto se ha decidido usar Django como soporte para estructura, entorno, tareas automatizadas y librerías, pero sin depender de su framework web en esta primera fase. Esto permite:

- Usar solo lo necesario de Django (por ejemplo, el ORM o settings).
- Mantener la herramienta como ejecutable CLI sin servidor web.
- Preparar la base para una futura migración completa a una app web si se desea.